NSTITUTO TECNOLÓGICO DE VERACRUZ

Ingeniería En Sistemas Computacionales

DOCUMENTACIÓN TÉCNICA INKBLACK
UNIDAD 2

Materia:

8J16A 9J12A - BASES DE DATOS NOSQL

Profesor(a):

JOSÉ NAHUM PRIETO

Alumnos:

- ALEXANDRA YAZMIN DOMÍNGUEZ NAMORADO
21020367

-LUIS ENRIQUE MACHORRO TRONCO
22020045

Veracruz,ver. 19 de octubre de 2026

Índice

INTRODUCCIÓN 3
MIGRACIÓN DE BASE DE DATOS 4
Proceso de Exportación 5
Proceso de Carga e Importación 5
SERVICIOS DE GITHUB STUDENT DEVELOPER PACK 7
SEGURIDAD: CONTROL DE ACCESO Y AUTENTICACIÓN 8
HARDENING DE RED Y POLÍTICAS DE FIREWALL 9
CONSULTAS CRUD NOSQL 10
DEPLOY 11
CONCLUSIONES 18

INTRODUCCIÓN

El presente documento técnico detalla el proceso de implementación, migración y aseguramiento de la infraestructura de base de datos para el proyecto inkblack. Como parte de la evolución tecnológica y optimización de la arquitectura del sistema, se llevó a cabo una transición estratégica desde un modelo relacional estricto (alojado previamente en Supabase) hacia un modelo NoSQL orientado a documentos utilizando la plataforma en la nube MongoDB Atlas. Este cambio de paradigma permite aprovechar la flexibilidad del esquema dinámico (BSON), mejorando la escalabilidad y el manejo de datos jerárquicos del proyecto.

El objetivo principal de este reporte, elaborado desde la perspectiva de la Jefatura de Proyecto, es evidenciar el cumplimiento integral de los requerimientos técnicos y de seguridad exigidos para el despliegue a producción. La documentación aborda la configuración de un entorno en la nube altamente protegido mediante la filosofía de "Defensa en Profundidad", abarcando la implementación de Control de Acceso Basado en Roles (RBAC), endurecimiento de red (Hardening) mediante listas de control de acceso y políticas CORS, así como el cifrado de datos en tránsito apoyado por certificados SSL.

Para fortalecer esta infraestructura, el proyecto integra servicios de grado industrial obtenidos a través del GitHub Student Developer Pack, tales como la gestión de dominios con Namecheap y bitácora operativa, exponiendo la estructura de las colecciones migradas, la ejecución de comandos administrativos y la sintaxis exacta de las operaciones de persistencia de datos (CRUD), garantizando un ecosistema de software mejorado, seguro y eficiente.

MIGRACIÓN DE BASE DE DATOS

Supabase (PostgreSQL) a MongoDB Atlas

El objetivo de este proceso fue trasladar la persistencia de datos de un modelo relacional estricto (Supabase / PostgreSQL) a un modelo de documentos flexible y escalable (MongoDB Atlas). Esto permite una mayor agilidad en el desarrollo y un manejo más eficiente de estructuras de datos dinámicas, eliminando la necesidad de esquemas rígidos (esquema *schema-less*).

SQL vs NoSQL

En esta fase se realizó el mapeo de las entidades relacionales a colecciones de documentos.

Entorno Origen (Supabase / Relacional)

- Paradigma: Base de datos relacional orientada a tablas, filas y columnas.
- Características: Tipos de datos estrictos, uso de llaves primarias (PK) y llaves foráneas (FK) para relacionar datos mediante uniones (JOIN).

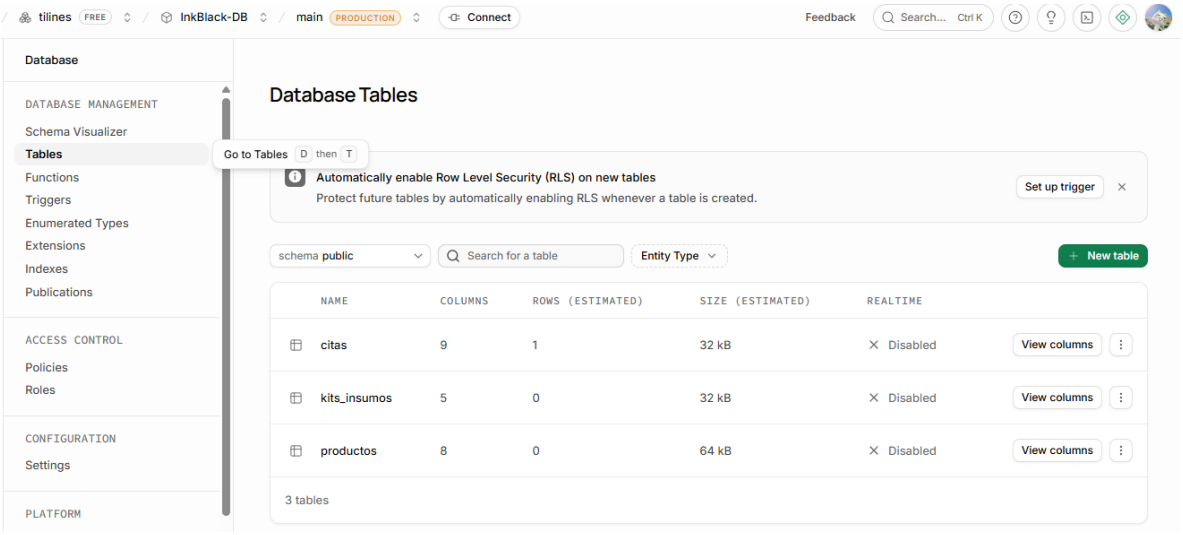


Imagen 1

Entorno Destino (MongoDB Atlas)

- Paradigma: Base de datos NoSQL orientada a documentos (formato BSON/JSON).
- Características: Colecciones en lugar de tablas, documentos en lugar de filas. Ausencia de llaves foráneas estrictas; las relaciones se manejan mediante referencias (ObjectIds) o documentos embebidos.

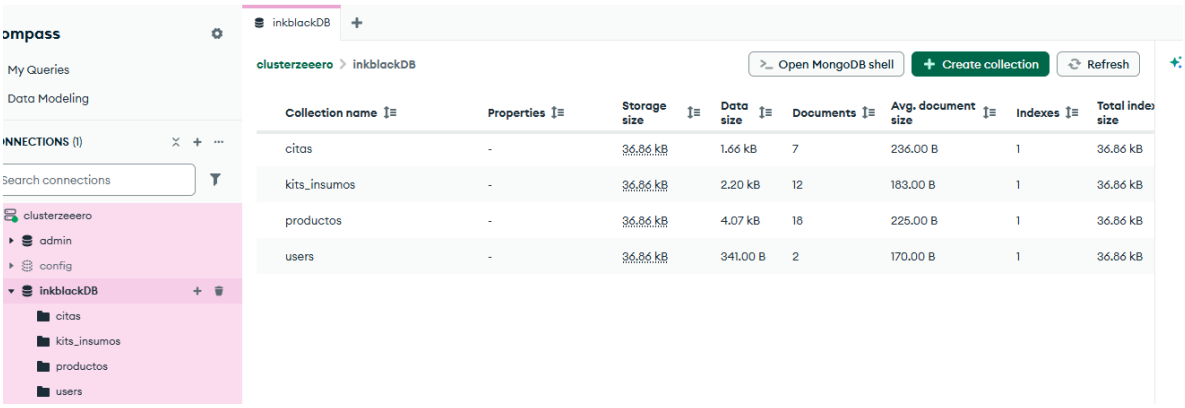


Imagen 2.

Proceso de Exportación

1. Se accedió al panel de control del proyecto en Supabase (Table Editor).
2. Se seleccionaron las tablas correspondientes a migrar.
3. Se utilizó la herramienta nativa de Supabase para exportar los registros existentes en formato **CSV**.

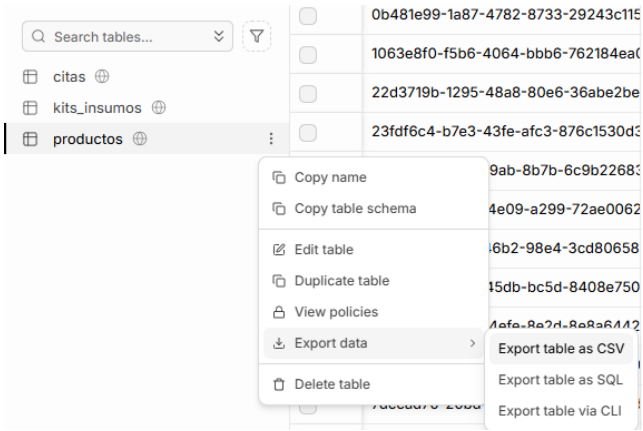


Imagen 3.

4. Se realizó una limpieza rápida de los datos exportados para asegurar que no hubiera caracteres corruptos y que los formatos de fecha fueran compatibles (formato ISO 8601).

Proceso de Carga e Importación

1. Se desplegó un nuevo clúster M0 (Sandbox) en **MongoDB Atlas**.
2. Se creó una nueva base de datos llamada inkblackDB

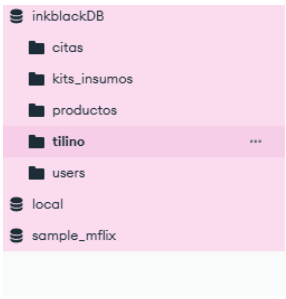
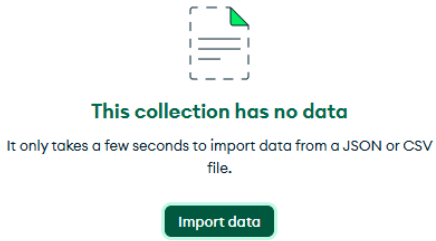


Imagen 4

3. Se definieron las colecciones principales correspondientes a las entidades del negocio.
4. A través de la interfaz de MongoDB Atlas (o utilizando la herramienta MongoDB Compass), se importaron los archivos CSV.



5. El motor de importación de MongoDB interpretó automáticamente las cabeceras de los archivos como las claves de los campos JSON y asignó un _id único (ObjectId) a cada registro nuevo.

Validación y Pruebas de Integridad

A continuación, se muestra la ejecución en consola utilizada para conectarse al clúster, autenticarse y verificar la base de datos migrada:

- Conexión segura por consola mongosh hacia el clúster de producción:

mongosh "mongodb+srv://clusterzeeero.wtee2lm.mongodb.net/" --apiVersion 1 --username l21020367_db_user

- Selección de la base de datos y comprobación de las estadísticas de migración:

use inkblack
show collections
db.stats()

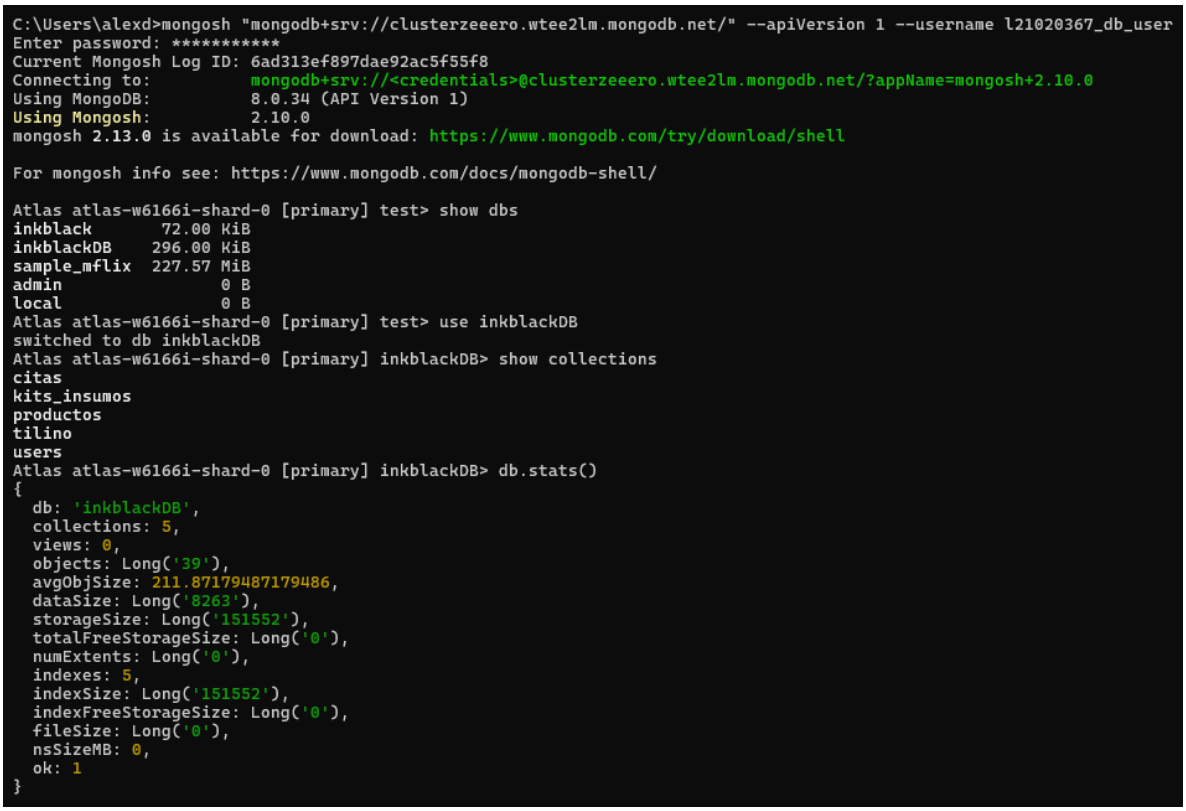


Imagen 5: Autenticación segura mediante MongoDB Shell (mongosh), verificación de colecciones migradas y métricas de almacenamiento de la base de datos de producción inkblackDB.

SERVICIOS DE GITHUB STUDENT DEVELOPER PACK

Para complementar y fortalecer la infraestructura y el desarrollo del proyecto **inkblack**, se activaron y configuraron servicios de grado profesional incluidos en el paquete de beneficios para estudiantes de GitHub:

- **MongoDB Atlas (Beneficio para Estudiantes):** Se utilizaron los beneficios del GitHub Student Pack para desplegar el clúster en la nube (clusterzeeero e inkblackDB). Esto permitió pasar de un entorno de desarrollo local a un entorno de producción (DBaaS - Database as a Service) con alta disponibilidad y monitoreo en tiempo real, alojando la base de datos inkblackDB.

- **GitHub Copilot:** Se integró la inteligencia artificial de GitHub Copilot en el entorno de desarrollo (IDE) del equipo. Esta herramienta fue fundamental para agilizar la refactorización del código backend y asistir en la sintaxis de las consultas de agregación BSON y operaciones CRUD nativas hacia MongoDB.

- **Namecheap (Gestión de Dominio y DNS):** Se utilizó el beneficio de Namecheap para registrar un nombre de dominio profesional para el proyecto. Mediante su panel de control, se configuraron los registros DNS para apuntar el dominio hacia el servidor donde se aloja el Frontend y la API del Backend. Esto permite que la plataforma sea accesible a través de una URL amigable, mejorando la presentación y arquitectura de red.

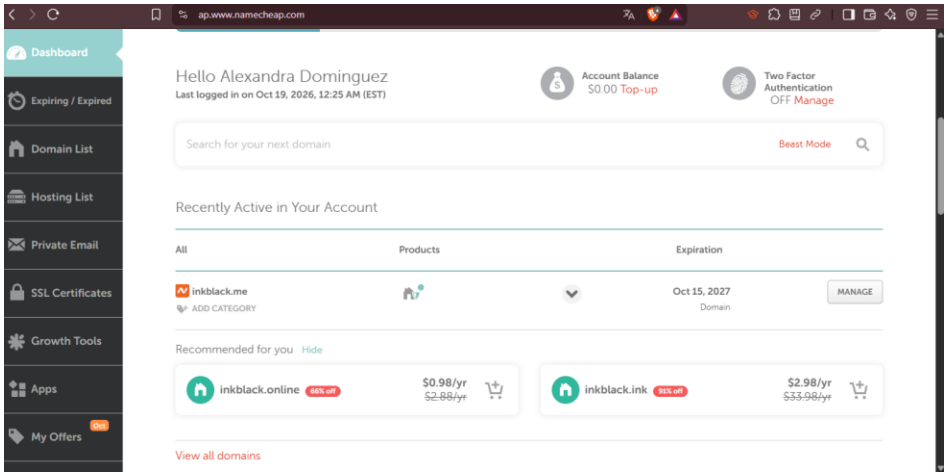


Imagen 6: Panel de Namecheap utilizado para la gestión del dominio del proyecto.

SEGURIDAD: CONTROL DE ACCESO Y AUTENTICACIÓN

Se programó un esquema de seguridad por capas para proteger las cuentas y la información del sistema **inkblack**:

- **Control de Acceso Basado en Roles (RBAC en Aplicación)** Cada usuario posee un atributo de rol (en este caso solo son dos Admin) dentro de su documento BSON en la colección users. Al iniciar sesión, el servidor valida este rol y genera un token criptográfico (JSON Web Token - JWT). El sistema verifica este token y el rol asociado en cada petición HTTP antes de permitir el acceso a rutas protegidas o ejecutar consultas CRUD.

- **Cifrado de Contraseñas con Bcrypt** Las contraseñas de todos los usuarios se almacenan procesadas mediante algoritmos de hash (Bcrypt). El sistema recibe la contraseña, genera el hash con un "salt" dinámico y lo compara con la base de datos durante el login. En ningún punto de la colección users se guardan contraseñas en texto claro, mitigando riesgos en caso de exposición de datos.

CIFRADO EN TRÁNSITO Y EN REPOSO

- **Cifrado en Tránsito** Toda la comunicación web se realiza forzosamente por HTTPS bajo protocolo TLS utilizando los certificados SSL gestionados a través de **Namecheap** en el dominio oficial del proyecto. A nivel de base de datos, la conexión entre el servidor y MongoDB Atlas viaja nativamente cifrada mediante el protocolo mongodb+srv://, impidiendo la intercepción de paquetes.

- **Cifrado en Reposo** Los datos de las 4 colecciones documentales (citas, kits_insumos, productos, users) alojadas en la base de datos inkblackDB se encuentran protegidos en disco en los servidores de MongoDB Atlas. Esto se logra mediante el motor de almacenamiento WiredTiger, el cual utiliza cifrado de grado industrial (AES-256) a nivel de volumen de almacenamiento.

HARDENING DE RED Y POLÍTICAS DE FIREWALL

Para mitigar el riesgo de ataques automatizados, escaneos de puertos y accesos no autorizados a la infraestructura de **inkblack**, se implementaron medidas de endurecimiento de red (Hardening) en múltiples capas:

- **Firewall Perimetral en MongoDB Atlas (IP Access List)** Se aplicó una política de denegación por defecto (Default Deny). Se eliminó la regla de acceso público global (0.0.0.0/0) en el panel de *Network Access* del clúster. En su lugar, se configuró una lista blanca (Whitelist) estricta, permitiendo únicamente el tráfico entrante proveniente de la dirección IP estática del servidor de producción y la red de desarrollo autorizada. Cualquier intento de

conexión desde una IP externa es bloqueado automáticamente a nivel de red, impidiendo ataques de fuerza bruta.

- **Gestión Segura de Variables de Entorno (.env)**

  Para la administración del entorno de producción alojado en Vercel y railway, se eliminó cualquier credencial dura (Hardcoded) o cadena de conexión (mongodb+srv://...) del código fuente. La seguridad del servidor se gestiona mediante variables de entorno encriptadas directamente en el panel de Environment Variables (Settings > Environment Variables). Esto garantiza que parámetros críticos, como la contraseña del clúster de Atlas y el JWT_SECRET para la firma de tokens, permanezcan aislados de los entornos locales y jamás queden expuestos en el historial de los repositorios de GitHub.

CONSULTAS CRUD NOSQL

La persistencia y manipulación de la información dentro del ecosistema **inkblack** se gestiona mediante el uso de operaciones CRUD nativas del modelo documental de MongoDB. A diferencia de las sentencias SQL tradicionales, el sistema interactúa con el clúster clusterzee pero procesando objetos en formato BSON.

Las consultas se ejecutan sobre las colecciones principales del proyecto (citas, kits_insumos, productos, users), utilizando filtros específicos, operadores y actualizaciones atómicas para satisfacer la lógica de negocio.

A continuación, se presenta la matriz técnica con los casos de uso reales, propósitos y la sintaxis exacta ejecutada en la consola mongosh:

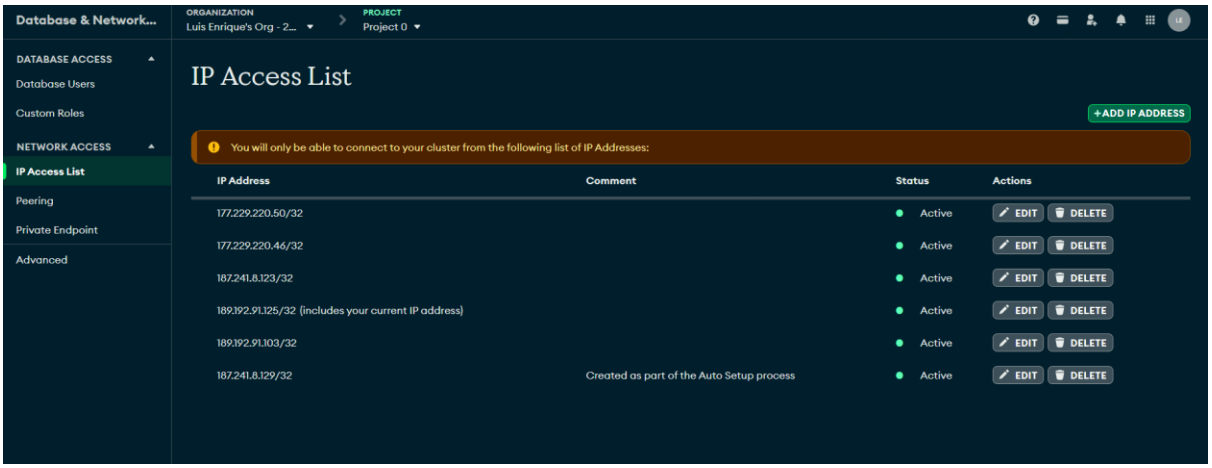
Operación / Método	Colección	Propósito en el Sistema	Sintaxis y Ejemplo de Ejecución (mongosh)
CREATE **insertOne()**	users	Registrar la cuenta de un nuevo usuario administrador en el sistema.	db.users.insertOne({ nombre: "Alexandra Domínguez", email: "alexandra@inkblack.me", rol: "admin", estado: "activo" })
CREATE **insertMany()**	productos	Insertar en bloque múltiples artículos e insumos al catálogo.	db.productos.insertMany([{ nombre: "Kit de Tatuaje Básico", stock: 12 }, { nombre: "Tinta Negra 1oz", stock: 50 }])
READ	citas	Consultar el listado de citas agendadas que se encuentran pendientes.	db.citas.find({ estado: "pendiente" }).pretty()

find()			
READ **find($lt)**	kits_insumos	Identificar y generar alerta de insumos críticos con inventario menor a 10.	db.kits_insumos.find({ stock: { $lt: 10 } })
READ **findOne()**	users	Buscar y extraer el documento de un usuario específico para el Login.	db.users.findOne({ email: "alexandra@inkblack.me" })
UPDATE **updateOne($set)**	citas	Cambiar atómicamente el estado de una cita de "pendiente" a "completada".	db.citas.updateOne({ _id: ObjectId("...") }, { $set: { estado: "completada" } })
UPDATE **updateOne($inc)**	productos	Descontar en 1 unidad el stock de un producto vendido sin reescribir todo el registro.	db.productos.updateOne({ nombre: "Tinta Negra 1oz" }, { $inc: { stock: -1 } })
DELETE **deleteOne()**	citas	Eliminar físicamente una cita que fue cancelada por el usuario.	db.citas.deleteOne({ _id: ObjectId("...") })
DELETE **deleteMany()**	tilino	Realizar depuración y limpieza masiva de registros de prueba (basura).	db.tilino.deleteMany({ tipo: "registro_prueba" })

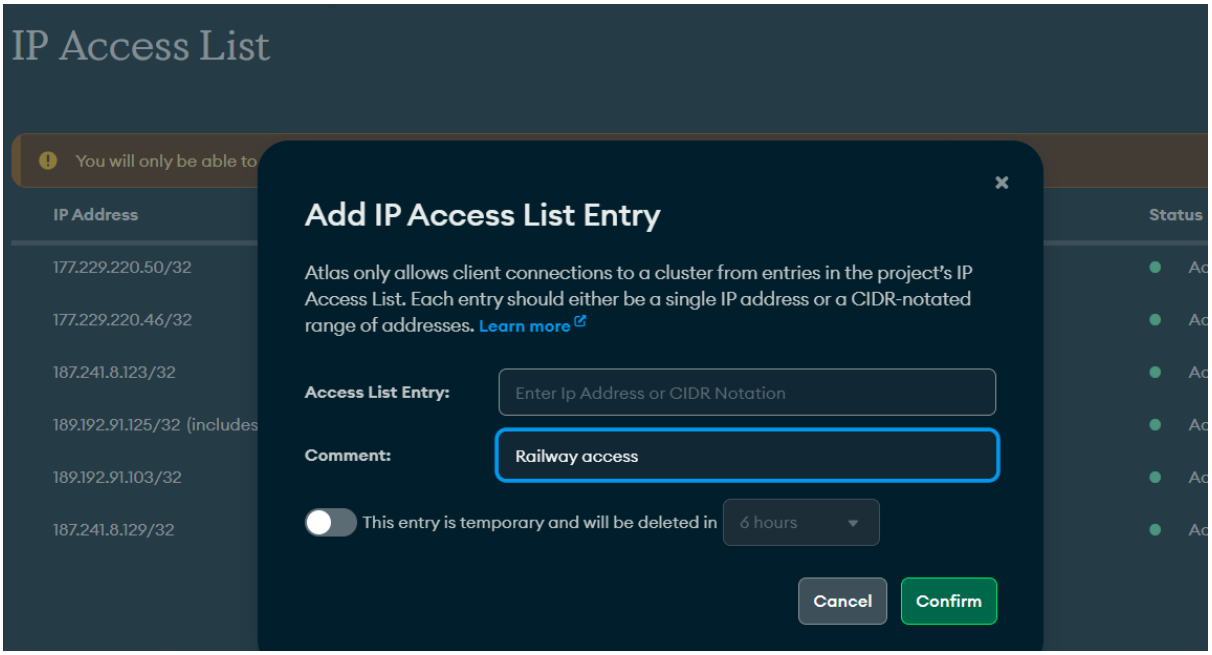
DEPLOY

El deploy (o despliegue de software) es el proceso de mover una aplicación, sitio web o servicio desde el entorno de desarrollo local (donde lo construyes y ejecutas en tu propia computadora) a un entorno de producción (servidores locales, servicios en la nube, contenedores o VPS) para que esté accesible y operativo para los usuarios finales o clientes.

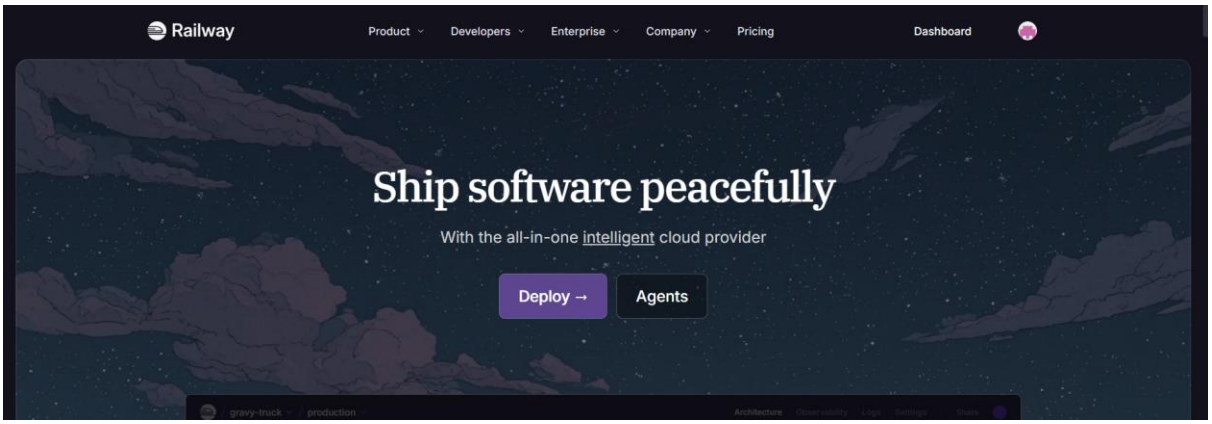
Para realizar el deploy debemos primero volver a nuestro MongoDB Atlas, para añadir nuestra IP dinámica y así la herramienta que usamos (en este caso: Railway).



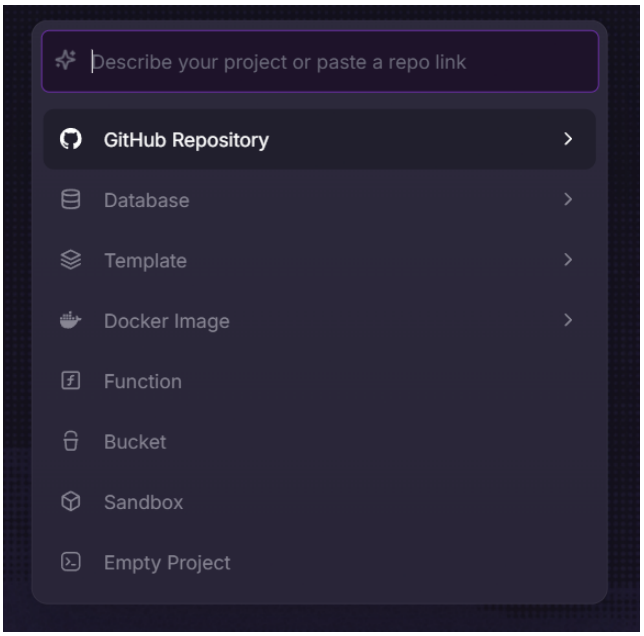
Aquí IP agregamos en el Access List Entry, que seria 0.0.0.0/0 y en comment asignamos el nombre Railway Access que es la herramienta que estamos usando y confirmamos, agregando así nuestra IP Access.



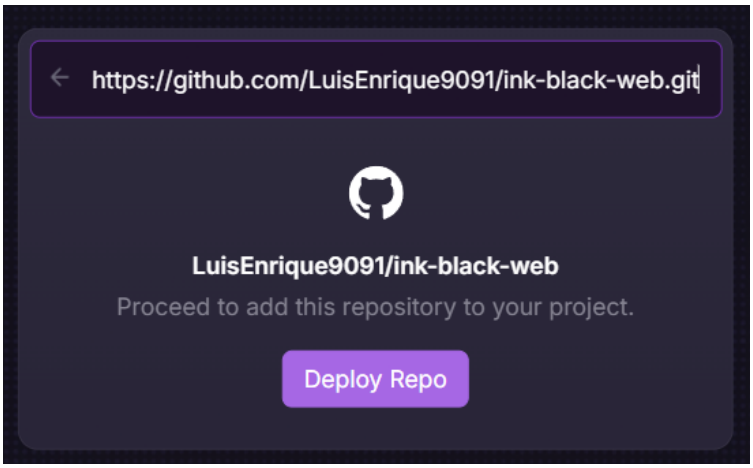
Ahora vamos a la aplicación, y seleccionamos la opción de “Deploy”, para esto tenemos que iniciar con nuestra cuenta de github.



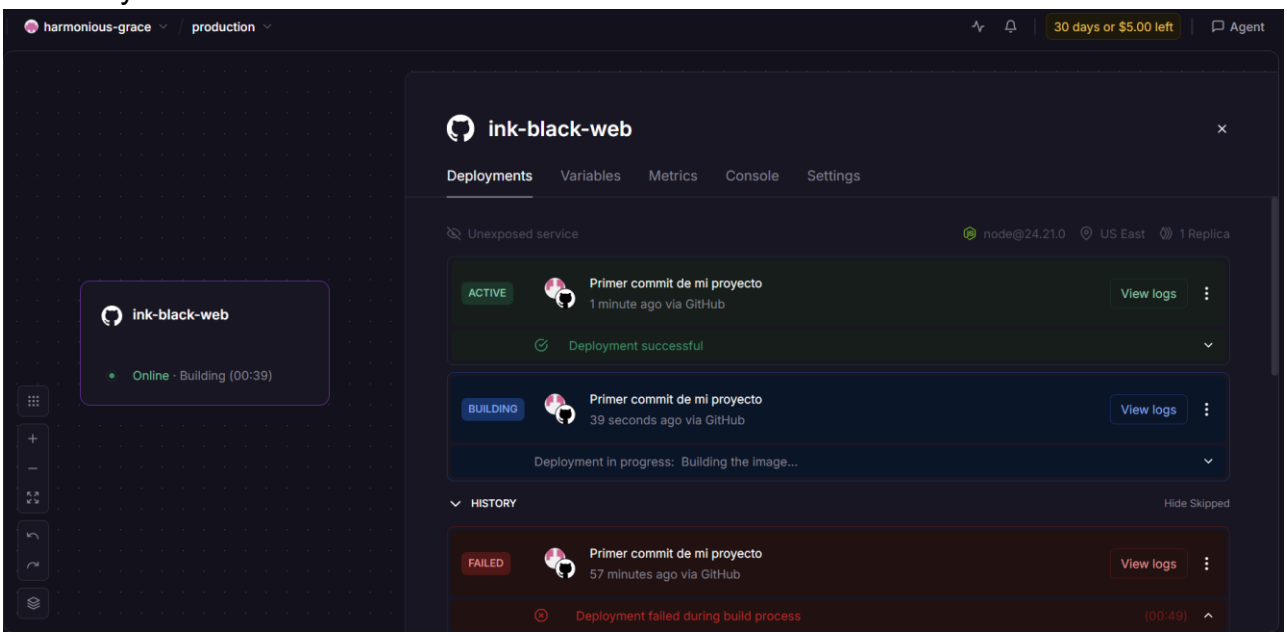
Nos redireccionará a un mensaje sobre si queremos agregar nuestro archivo desde un repositorio en github y copias la URL que nos aparece en el “Code” de GitHub y lo pegamos para que encuentre el repositorio.



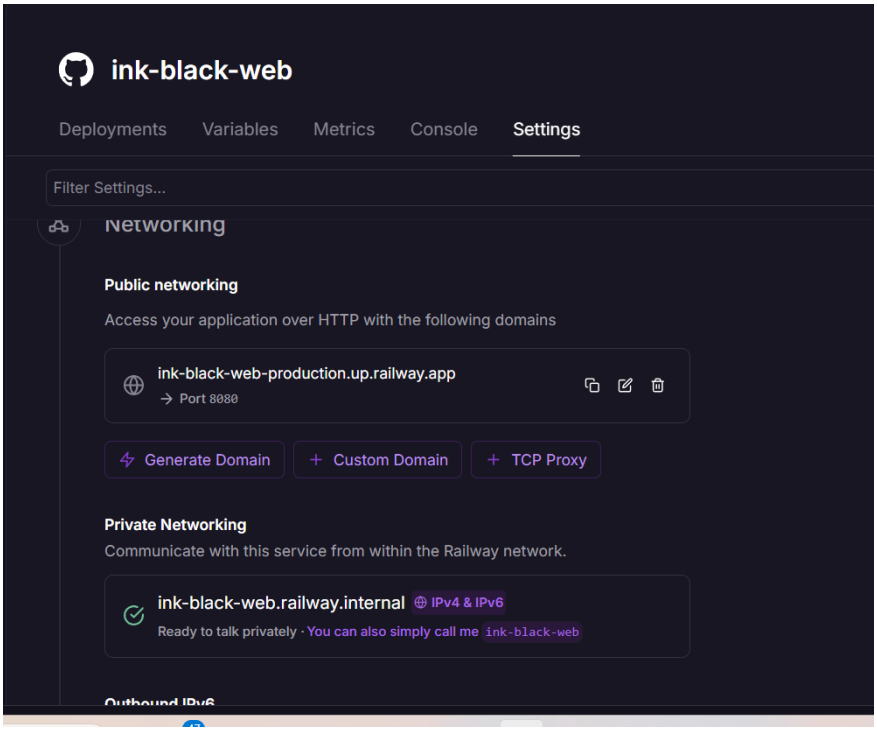
Una vez que lo encuentre solo daremos clic en Deploy Repo



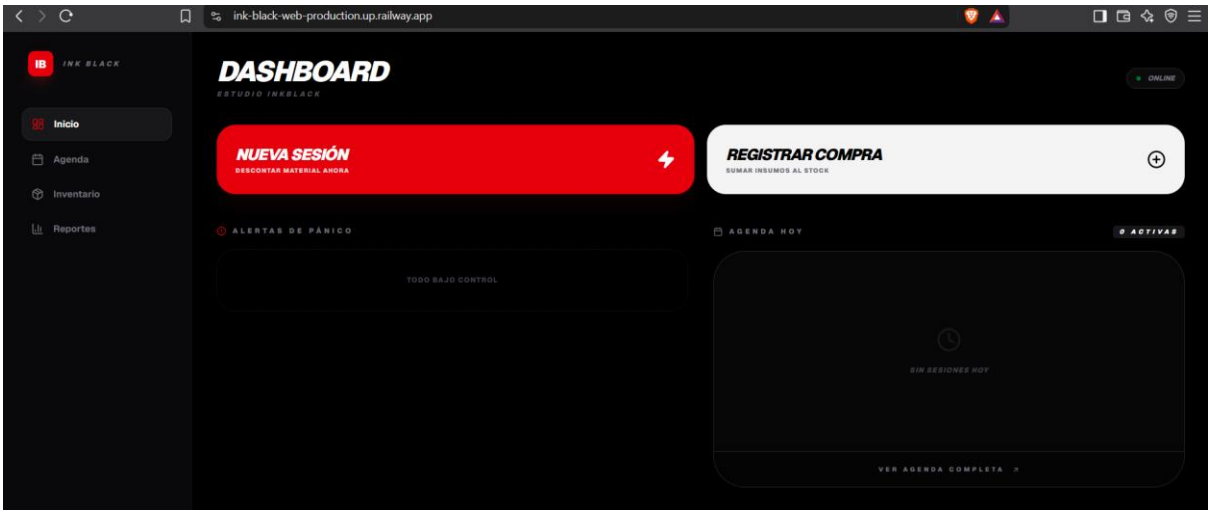
Aquí cargará nuestros archivos y analizará si hay algún error en ellos en este caso subo la captura con los archivos ya solucionados y por último se subió completo el análisis y se subió.



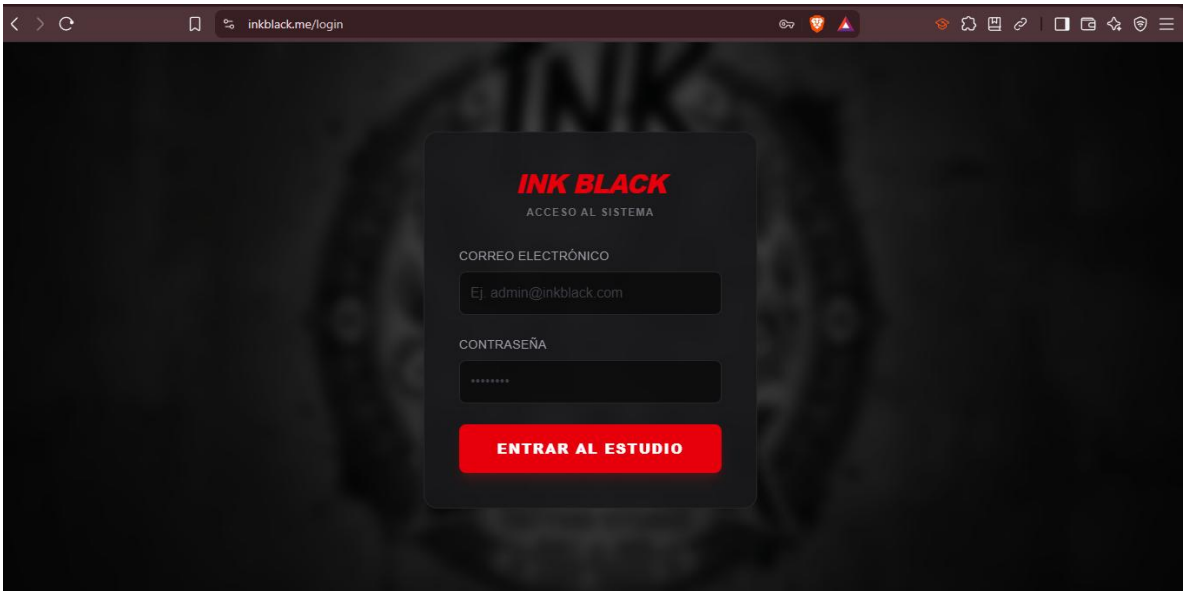
iremos al apartado de settings y bajaremos a Networking ahi estara el URL de la página realizada, solo queda pegarla en el buscador.



Y por último entraremos a la página donde nos mostrara nuestro trabajo realizado ahora ya en línea dejando de ser local



Al conectar el dominio inkblack.me podemos verificar que la página funciona correctamente:



CATÁLOGO DE VISTAS DEL SISTEMA WEB

El sistema **inkblack** cuenta con una interfaz gráfica (Front End) con temática oscura, conectada en tiempo real a la base de datos NoSQL. A continuación, se documentan las vistas principales programadas y su interacción con MongoDB Atlas:

1. **Vista de Acceso al Sistema (Login):**

Interfaz de autenticación principal donde se solicita "Correo Electrónico" y "Contraseña" para "ENTRAR AL ESTUDIO".
Colección vinculada: users.
Operación: Ejecuta una lectura (findOne) para validar las credenciales. Al coincidir, autoriza el acceso al resto de los módulos.

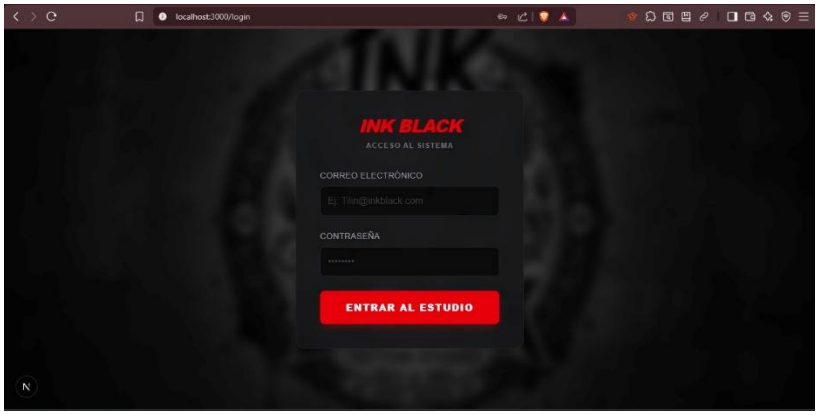


Imagen : Interfaz de acceso seguro (Login) de inkblack, conectada a la colección de usuarios para la validación de credenciales y generación de tokens de sesión.

2. **Vista Dashboard (Página Principal):**

Panel de control gerencial que muestra botones de acceso rápido ("NUEVA SESIÓN" y "REGISTRAR COMPRA"). Destaca un módulo crítico llamado **"ALERTAS DE PÁNICO"** que notifica automáticamente sobre stocks bajos (ej. "Quedan 4 en TINTAS") o insumos a punto de caducar. También incluye una vista rápida de la "AGENDA HOY".
Colecciones vinculadas: citas, kits_insumos.
Operación: Consultas condicionales avanzadas para detonar las alertas visuales de color rojo.

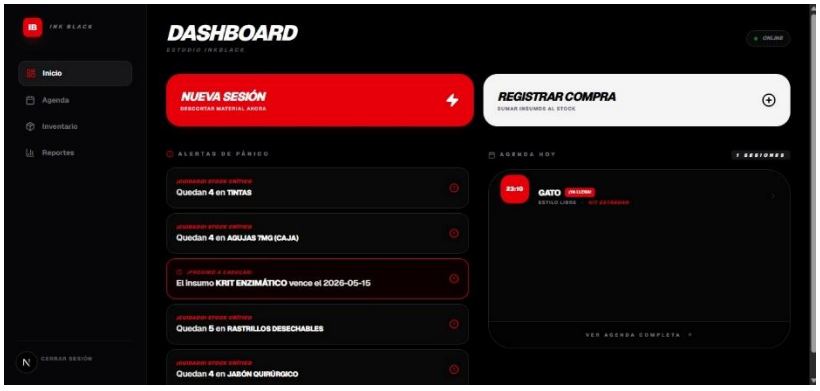


Imagen : Panel de Control principal (Dashboard), destacando el sistema automatizado de "Alertas de Pánico" que monitorea el inventario crítico en tiempo real.

3. **Vista de Gestión de Almacén (Inventario):**

Tabla técnica que lista todos los insumos mostrando su estado ("SUFICIENTE" o "COMPRAR PRONTO"), cantidad en stock y fecha de caducidad.
Módulos interactivos (Modales): Cuenta con ventanas emergentes para crear un **"NUEVO INSUMO"** desde cero (ingresando nombre, categoría, stock inicial y fecha), y otro modal para **"SURTIR ALMACÉN"** sumando existencias a un producto ya registrado.
Colección vinculada: kits_insumos.
Operaciones: Inserción de nuevos documentos (insertOne) y actualización matemática mediante incrementos para surtir mercancía (updateOne con operador $inc).

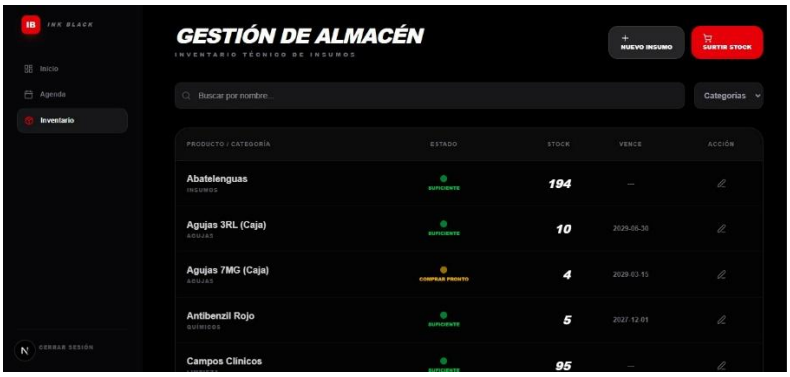


Imagen : Módulo de Gestión de Almacén, visualizando el cruce de datos de la colección de insumos con indicadores visuales de estado y caducidad.

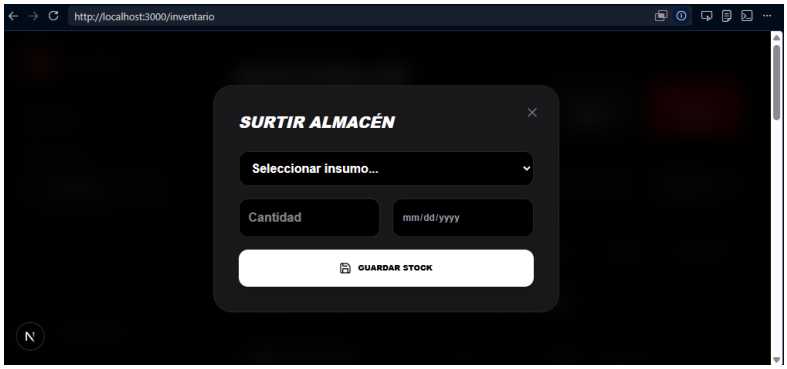


Imagen 5: Interfaz de actualización de stock, diseñada para ejecutar incrementos matemáticos atómicos (Operación UPDATE) sobre el inventario existente.

4. **Vistas de Agenda y Nueva Sesión:**

El módulo "Próximas Sesiones" lista el historial de clientes, indicando la hora, estilo de tatuaje y los estados de la cita mediante etiquetas visuales ("CONFIRMADA" en

verde, "CANCELADA" en gris). El módulo "Nueva Sesión" permite capturar un nuevo cliente, eligiendo mediante botones el kit a utilizar (KIT ESTÁNDAR, KIT MICRO, KIT REALISMO).

Colecciones vinculadas: citas y productos.

Operaciones: Inserción de nuevas citas (insertOne), modificación de los estados (updateOne), y eliminación física de registros descartados (deleteOne).

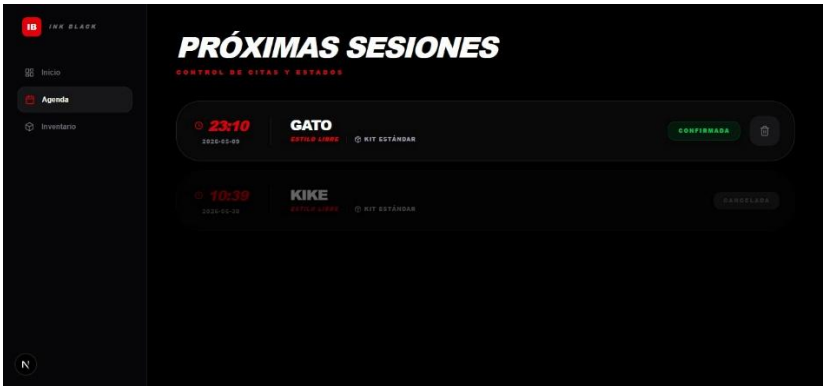


Figura 6: Historial de próximas sesiones, mostrando el control de estados de las citas (Confirmadas y Canceladas) extraídas de MongoDB.

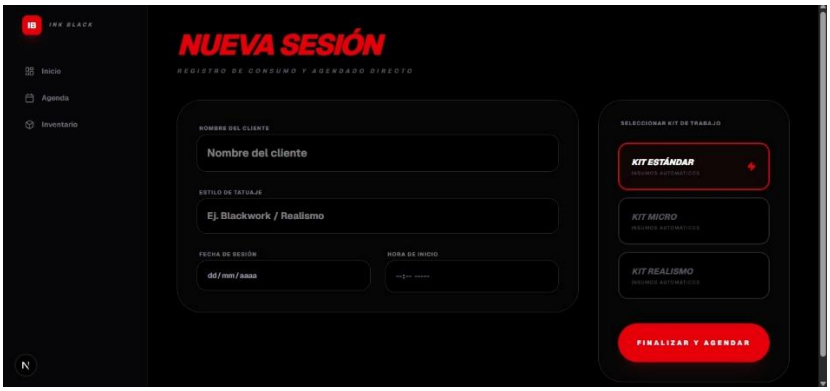


Figura 7: Formulario de agendado de citas, integrando la selección de kits de trabajo que permite el descuento automatizado de insumos por sesión.

CONCLUSIONES

La modernización de la arquitectura de datos de nuestro proyecto inkblack representó un salto cualitativo en la escalabilidad, rendimiento y seguridad del sistema. Como equipo, la migración estratégica desde un entorno relacional hacia el modelo NoSQL de MongoDB Atlas nos permitió eliminar la rigidez de las tablas tradicionales, facilitando el manejo de información dinámica y jerárquica mediante

colecciones de documentos BSON perfectamente adaptables a las necesidades del estudio.

En materia de ciberseguridad, logramos implementar con éxito una arquitectura de "Defensa en Profundidad". A nivel de aplicación, establecimos un Control de Acceso Basado en Roles (RBAC) garantizando el principio de menor privilegio; a nivel de red, aplicamos un endurecimiento (Hardening) mediante listas de acceso IP restrictivas en el Firewall de Atlas; y a nivel de transporte, aseguramos la confidencialidad de los datos mediante cifrado de extremo a extremo con protocolos TLS/SSL.

El valor tecnológico de nuestro desarrollo se vio altamente potenciado por la integración colaborativa de herramientas del GitHub Student Developer Pack, destacando la gestión de nuestro dominio oficial a través de Namecheap y la optimización del código con GitHub Copilot. A esto sumamos la arquitectura de despliegue en la nube, utilizando Vercel y Railway para alojar el Front End y ~~Render~~ para la API del Backend, logrando que nuestra interfaz gráfica interactúe en tiempo real con las operaciones de la base de datos de manera fluida y segura.

En resumen, la implementación integral de este proyecto demuestra nuestra capacidad como equipo para transicionar de un entorno de desarrollo académico a una infraestructura en la nube con verdaderos estándares de producción profesional, entregando en *inkblack* una plataforma segura y completamente funcional.